

Conditioning, Identifiability, and Regularization in an Adjoint-Based PDE Inverse Problem

Sipan Bareyan

June 21, 2026

1 Goal

In this report we study the inverse problem of recovering a potential V in a nonlinear PDE from a target state u_0 . We analyze the conditioning of the problem, behaviour of the Hessian and solvers for different cases. We discuss optimization strategies and the challenges that arise from the ill-posedness of the problem, exploring the conditioning of the Hessian, the identifiability of the potential, and the role of regularization in obtaining meaningful solutions.

2 Problem setup

We consider the nonlinear PDE

$$-u'' + \alpha u^3 + Vu = f, \quad x \in [0, 2\pi], \quad u(0) = u(2\pi), \quad u'(0) = u'(2\pi),$$

where u is the state, V the potential to be recovered, α a nonlinearity parameter, and f a source term. We recover V from a given target state u_0 by minimizing the misfit functional

$$I(V) = \int_0^{2\pi} (u(V) - u_0)^2 dx,$$

where $u(V)$ is the solution of the PDE for the potential V .

We use two instances: a well-conditioned one with $u_{\text{well}} = 2 + \sin x$ (strictly positive) and an ill-conditioned one with $u_{\text{ill}} = \sin x$ (sign-changing). The data is exact: for each instance we fix a true potential $V_\star$ and define the source term directly from the forward PDE,

$$f = -u_0'' + \alpha u_0^3 + V_\star u_0,$$

so that u_0 solves the PDE for $V_\star$ exactly. We then run the inverse problem and measure the recovered V against $V_\star$ (Figure 1).

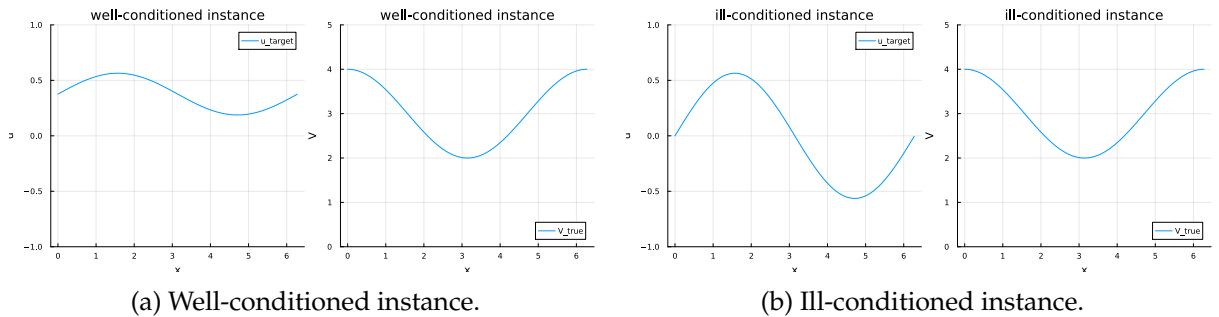


Figure 1: Target state u_0 and true potential $V_\star$ for the two instances used throughout.

3 Adjoint gradient and exact Hessian

To run a gradient-based optimizer we need the gradient of the misfit functional $I(V)$ with respect to V . As shown in Report 1, the adjoint method computes this gradient efficiently: the adjoint equation, derived from the forward PDE, yields the gradient without explicitly differentiating the solution $u(V)$ with respect to V . In our experiments we compute gradients with Zygote.jl automatic differentiation together with hand-written differentiation rules, which proved the most efficient and general method for our problem.

We now make this concrete and push it one step further to the Hessian, the key object for the solvability and identifiability of the inverse problem. Discretize on N equispaced points with quadrature weight $w = 2\pi/N$, and write the forward residual and the misfit as

$$\mathcal{F}(u, V) = -u'' + \alpha u^3 + V \odot u - f = 0, \quad I(V) = w \|u(V) - u_0\|_2^2.$$

Linearizing $\mathcal{F}$ about the state gives two symmetric operators, the tangent operator and the control sensitivity,

$$L := \mathcal{F}_u = -\partial_{xx} + \text{diag}(3\alpha u^2 + V), \quad \mathcal{F}_V = \text{diag}(u).$$

Differentiating the constraint $\mathcal{F}(u(V), V) = 0$ yields the state sensitivity

$$J := \frac{\partial u}{\partial V} = -L^{-1} \text{diag}(u).$$

Introducing the adjoint state λ defined by the single linear solve $L\lambda = u - u_0$, the chain rule collapses to

$$\nabla I = 2w J^\top (u - u_0) = -\frac{4\pi}{N} \text{diag}(u) \lambda,$$

so the gradient costs one forward and one adjoint solve, independent of $\dim V$.

Closed-form Hessian. Differentiating ∇I once more and contracting the second-order state sensitivity against the adjoint (using $u - u_0 = L\lambda$) gives the exact Hessian

$$H = \frac{4\pi}{N} \left(\underbrace{J^\top J}_{\text{Gauss-Newton}} + S \right), \quad S = -6\alpha J^\top \text{diag}(u \odot \lambda) J - J^\top \text{diag}(\lambda) - \text{diag}(\lambda) J.$$

The correction S is symmetric and available in closed form precisely because the only nonlinearity is cubic, so $\mathcal{F}_{uu} = 6\alpha \text{diag}(u)$ is diagonal and constant. Our data are exact, so at the minimizer the residual vanishes; then $u - u_0 \rightarrow 0 \Rightarrow \lambda \rightarrow 0 \Rightarrow S \rightarrow 0$, and the Hessian reduces to its Gauss-Newton form

$$H = \frac{4\pi}{N} J^\top J = \frac{4\pi}{N} \text{diag}(u) L^{-2} \text{diag}(u).$$

This last expression drives the entire analysis: it is a smoothing kernel L^{-2} modulated on both sides by the state u , and we read the conditioning (Section 4.1) and the identifiability of V directly off it.

4 Analysis of numerical efficiency

4.1 Conditioning and identifiability

We visualize the Hessian H at the true potential both in position space (Figure 2) and through its spectrum (Figure 3). The reading follows directly from $H = \frac{4\pi}{N} \text{diag}(u) L^{-2} \text{diag}(u)$, whose (i, j) entry is $\frac{4\pi}{N} u_i (L^{-2})_{ij} u_j$.

For the well-conditioned instance ($u_{\text{well}} = 2 + \sin x > 0$) the modulation $\text{diag}(u)$ is invertible, so H inherits the full rank of L^{-2} and its spectrum decays smoothly. The decay is itself meaningful: L^{-2} damps high-frequency modes of V , so these directions are only weakly constrained by the data, but none of them is lost. Every direction in V is identifiable.

For the ill-conditioned instance the spectrum drops off a cliff to machine zero — a genuine null space, not a numerical artifact. The mechanism is the modulation by u . The target $u_{\text{ill}} = \sin x$ vanishes at $x = 0$ and $x = \pi$, so the corresponding rows and columns of H are identically zero; these are the dead cross-bands punched through the smoothing kernel in Figure 2b. Equivalently, the adjoint gradient $-\frac{4\pi}{N} (u \odot \lambda)$ carries a factor of u and cannot move V wherever $u = 0$. Along these directions the misfit is exactly flat: the data say nothing about V at the zeros of u , and the problem is non-identifiable there. (The recovered V that exposes this is shown after the optimizer test, Figure 7.)

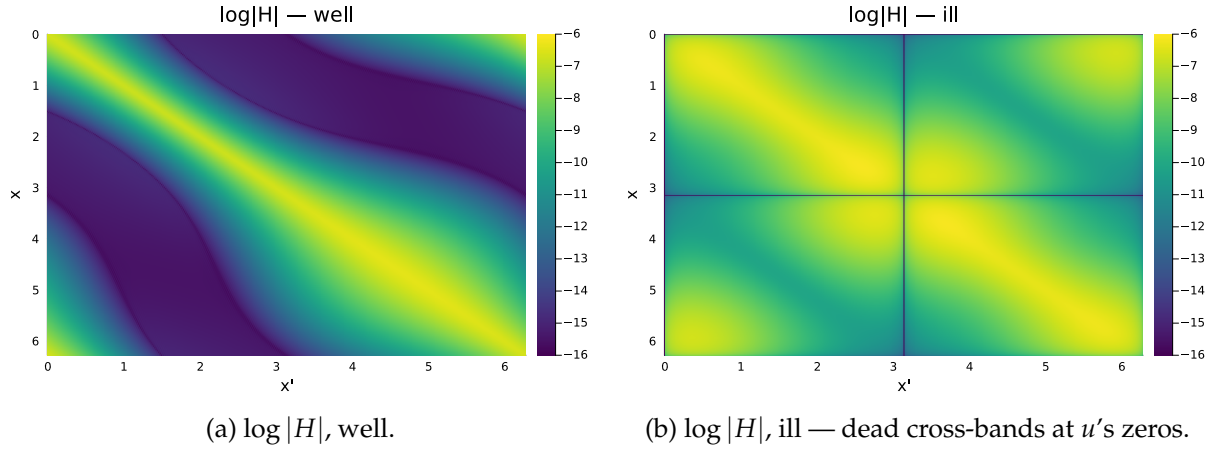


Figure 2: Hessian magnitude in position space. The band is the L^{-2} smoothing kernel; the ill case is punched through at $x = 0, \pi$ — the null directions.

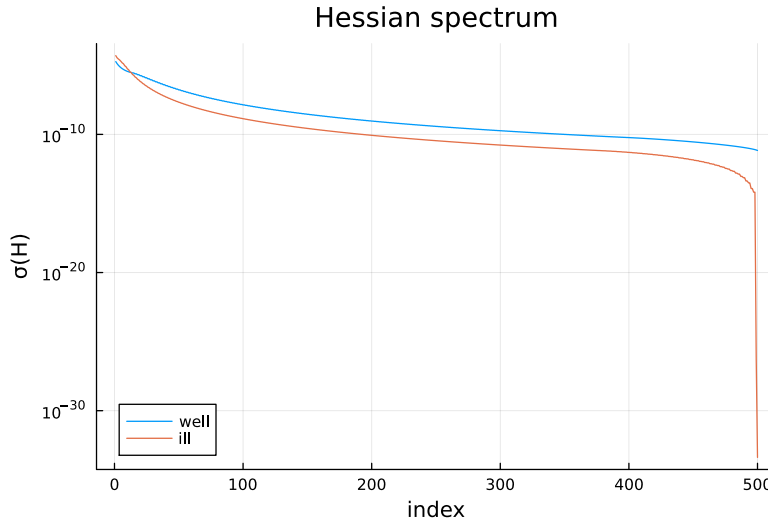


Figure 3: Hessian spectrum for both instances: a shared smoothing decay, plus a cliff to machine zero in the ill case.

config	κ	pot_err
well	2.72394e6	6.7143e-8
ill	1.21785e29	0.0792048

Table 1: Condition number κ and recovery error per instance.

4.2 Dependence on nonlinearity and resolution

Two knobs change the problem without changing its character: the nonlinearity parameter α and the mesh resolution N . They act on different parts of $H = \frac{4\pi}{N} \text{diag}(u) L^{-2} \text{diag}(u)$, so we sweep them separately.

4.2.1 Nonlinearity α

The parameter α enters H only through $L = -\partial_{xx} + \text{diag}(3\alpha u^2 + V)$, a bounded perturbation of the diagonal. It does not create or remove zeros of u , which is what controls the rank. We therefore expect the condition number κ to be largely insensitive to α , and that is exactly what the sweep shows (Figure 4, Tables): the well-conditioned instance stays well-conditioned and the ill-conditioned instance stays singular across the whole range. Increasing the nonlinearity does not change the qualitative nature of either problem.

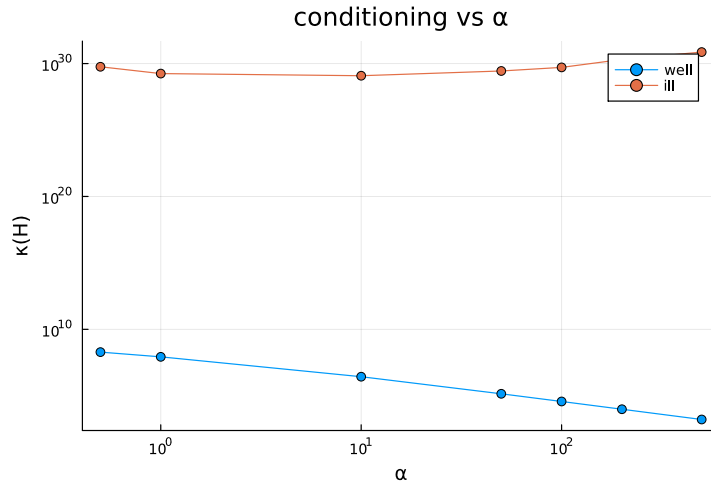


Figure 4: Condition number κ versus nonlinearity α , both instances. Each curve is essentially flat: κ barely responds to α .

α	newton	κ	it_to_tol	pot_err	α	newton	κ	it_to_tol	pot_err
0.5	7	1.93359e8	35	3.36685e-8	0.5	4	5.75522e29	-1	0.0974227
1.0	11	8.52891e7	24	2.86324e-8	1.0	5	1.76521e29	-1	0.0940876
10.0	14	2.72394e6	14	6.7143e-8	10.0	8	1.21785e29	-1	0.0792048
50.0	18	1.40992e5	14	2.57637e-7	50.0	12	2.77259e29	2746	0.0676878
100.0	22	37242.4	14	1.31564e-6	100.0	13	5.159e29	1621	0.06716
200.0	21	9713.31	13	4.33682e-6	200.0	15	2.03784e30	1000	0.0690864
500.0	24	1643.71	13	4.0546e-5	500.0	17	7.33613e30	496	0.0709722

α -sweep, well-conditioned

α -sweep, ill-conditioned

Table 2: α -sweep conditioning data for both instances.

4.2.2 Resolution N

Resolution acts on the smoothing kernel L^{-2} itself. The eigenvalues of $L \approx -\partial_{xx}$ range from $O(1)$ for the smoothest mode up to $O(N^2)$ for the highest frequency the grid can represent, so the eigenvalues of L^{-2} range from $O(1)$ down to $O(N^{-4})$. For the well-conditioned instance $\text{diag}(u)$ is a bounded, invertible modulation that does not change this scaling, hence

$$\kappa(H) \sim N^4,$$

the clean quartic growth seen in Figure 5. This is intrinsic ill-posedness rather than a discretization defect: refining the mesh exposes ever finer, ever less constrained modes of V . For the ill-conditioned instance the null space already makes κ effectively infinite (numerically bounded only by round-off); it does not follow the N^4 law and does not settle, oscillating at a high value as the machine-zero floor shifts with N .

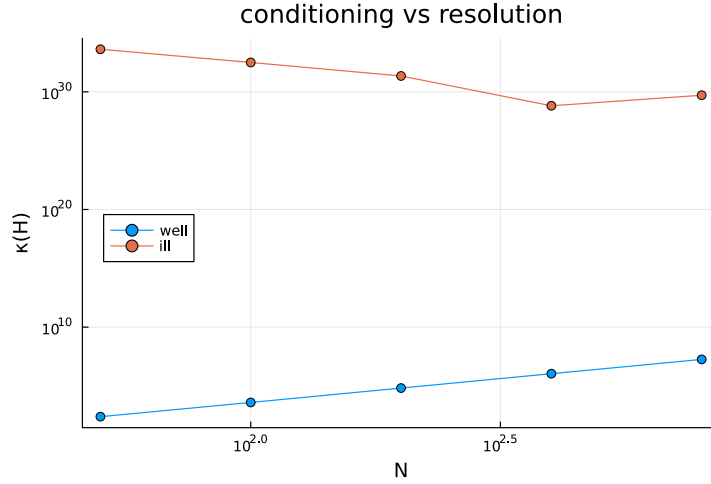


Figure 5: Condition number κ versus resolution N . The well case grows as N^4 ; the ill case stays high and irregular.

N	κ_{well}	κ_{ill}
50	246.792	4.14802e33
100	3977.55	3.11929e32
200	66524.8	2.22864e31
400	1.10511e6	6.52285e28
800	1.81481e7	5.11427e29

Table 3: N -sweep (resolution) conditioning data.

5 Optimizer comparison: the identifiability gap

We compare three optimizers: gradient descent (GD), Adam, and L-BFGS. Gradient descent is the simplest method. Adam is more sophisticated, using adaptive learning rates and momentum to accelerate convergence; it is designed primarily for machine-learning problems but outperforms GD in most settings. L-BFGS is a quasi-Newton method that approximates the inverse Hessian, and is particularly effective on ill-conditioned problems because it adapts to the local curvature.

We compare the optimizers on both instances. First, we run each optimizer for a fixed iteration budget and track the loss and the potential error $\|V - V_\star\|/\|V_\star\|$ at every iteration (Figure 6).

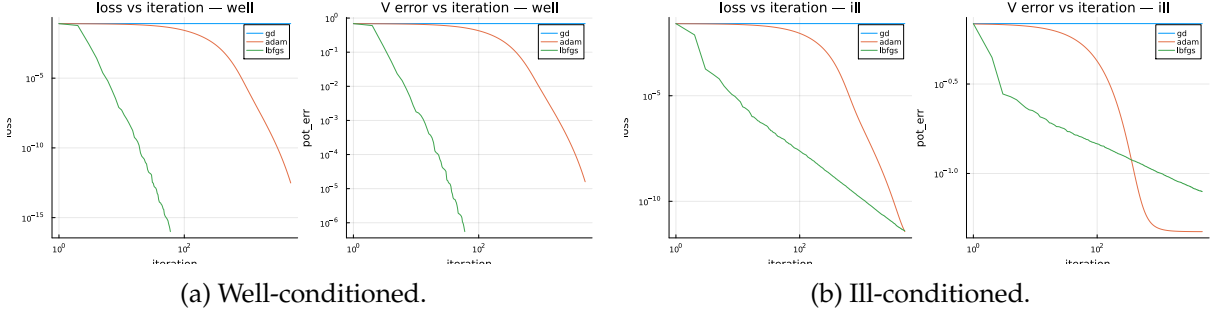


Figure 6: Loss (left) and potential error $\|V - V_\star\|/\|V_\star\|$ (right) vs. iteration, for the well-conditioned (left) and ill-conditioned (right) instances. On the ill case the loss keeps falling while the potential error stalls.

Second, we record how many iterations and how much wall-clock time each optimizer needs to reach a target loss, together with the final loss and recovery error (Table 4).

config	optimizer	it_to_target	s_to_target	it_to_tol	final_loss	pot_err	total_s
well	gd	-1	NaN	-1	0.0103494	0.68793	212.711
well	adam	653	39.9147	1236	1.6278e-17	7.97524e-8	143.277
well	lbfgs	7	0.782323	14	1.18994e-17	6.7143e-8	2.23521
ill	gd	-1	NaN	-1	0.0260746	0.687421	70.4621
ill	adam	864	8.98792	-1	3.65145e-12	0.0472864	52.0134
ill	lbfgs	18	0.346995	-1	3.99795e-12	0.0792048	96.4067

Table 4: Iterations and wall time to the target/converged loss, final loss, and recovery, per optimizer.

On the well-conditioned instance every method that moves at all recovers V : L-BFGS converges fastest and reaches the lowest loss (a few iterations, well under a second), while Adam is slower but still drives both the loss and the potential error to zero. Gradient descent, by contrast, never makes progress — its curves are flat (Figure 6). This is a conditioning-driven failure rather than a coding one: with $\kappa(H)$ as large as it is (Section 4.2), a single fixed step size cannot move the stiff and the soft modes at once, so the iterate is effectively frozen.

The ill-conditioned instance exposes the central point. Here the loss still falls to near machine zero, yet the potential error *stalls* — minimising the misfit no longer recovers V . The ranking even inverts: L-BFGS reaches the lowest loss but the *worst* potential ($\|V - V_\star\|/\|V_\star\| \approx 0.087$ in Table 4), whereas Adam settles at a higher loss but a *better* potential (≈ 0.047). A lower loss therefore does not imply a better recovery. The cause is structural, not algorithmic: the misfit is flat along the null directions of the Hessian (Section 4.1), so an optimiser is free to place arbitrary mass there. Figure 7 makes this concrete — the recovered V matches $V_\star$ everywhere except for spikes at $x = 0, \pi$, exactly the zeros of u where $\text{diag}(u)$ is singular and the data cannot constrain V . Fixing this needs a prior, not a better optimiser — the subject of Section 6.

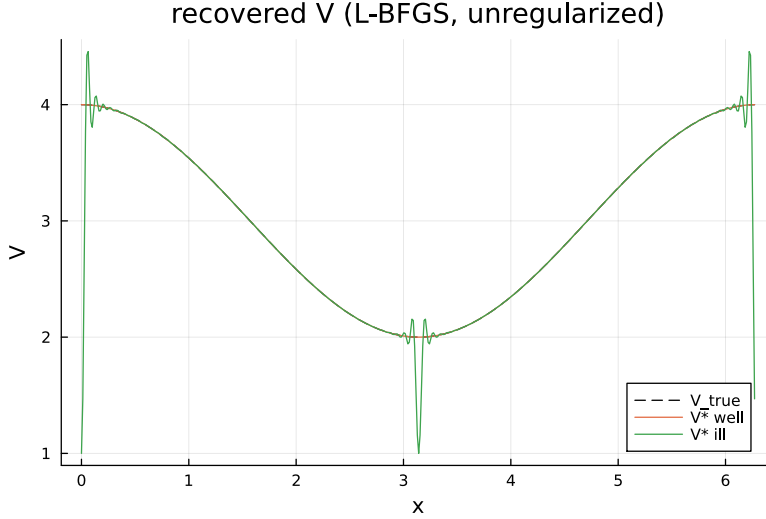


Figure 7: Recovered V (L-BFGS, unregularised): the ill-case error concentrates at the null points $x = 0, \pi$, where the data cannot constrain V .

6 Regularization

Regularization addresses several distinct problems at once. It stabilizes the inversion when the data u_0 are measured with noise; it injects prior knowledge about the potential, such as smoothness or sparsity; and it removes the ambiguity along the null directions of the Hessian by penalizing the otherwise-free modes, so that V can be recovered even where the data are blind to it.

The need is sharpest under noise. We perturb the data u_0 by an amount of order 10^{-6} — invisible to the naked eye (Figure 8, right) — and attempt the naive pointwise inversion that solves the PDE algebraically for V ,

$$V_{\text{direct}} = \frac{f + u'' - \alpha u^3}{u}.$$

This differentiates the noisy u twice, and the u'' term amplifies a Fourier mode of wavenumber k by a factor k^2 . The high-frequency noise is blown up and the reconstruction is destroyed (Figure 8, left). The variational recovery V^* , on the other hand, never differentiates the data and tracks V_* faithfully — which is why optimisation, not algebra, is the right tool here.

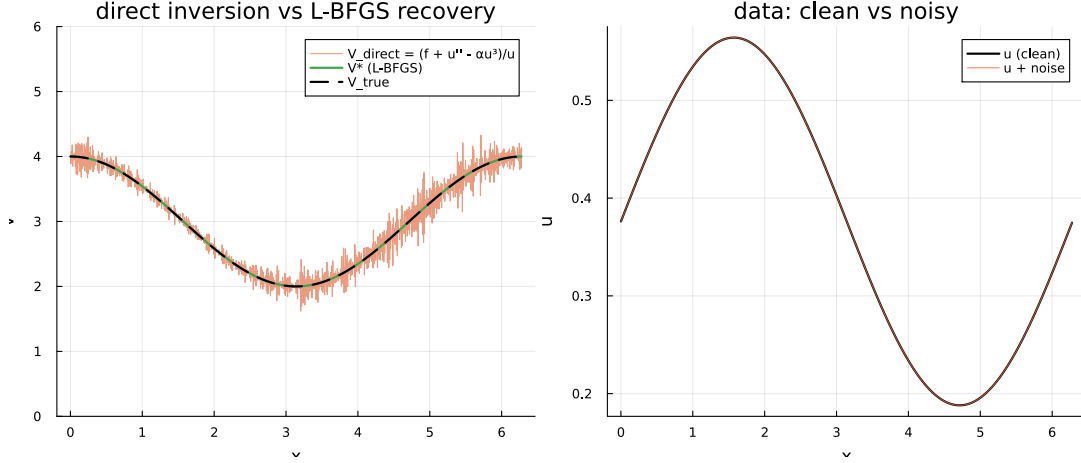
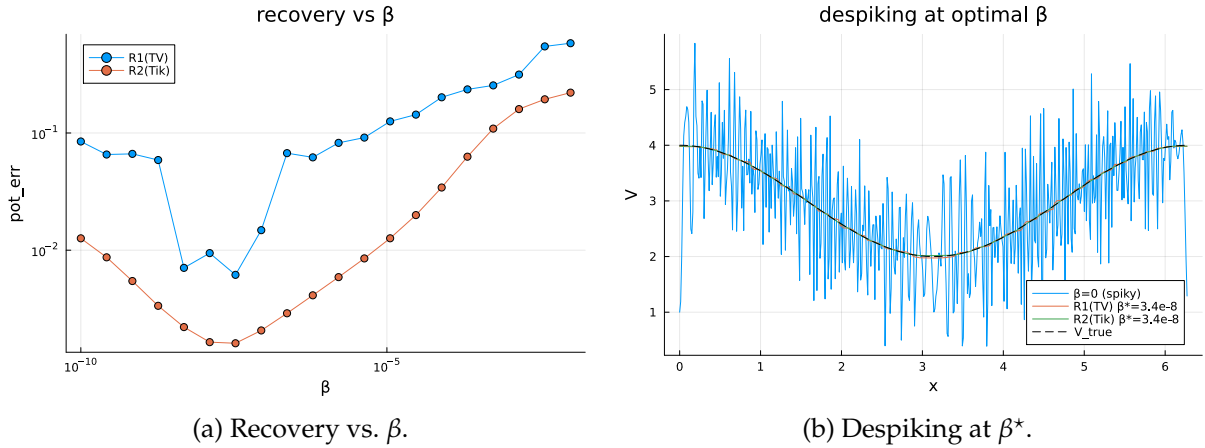


Figure 8: Why optimisation is needed. Left: the pointwise algebraic inversion $V_{\text{direct}} = (f + u'' - \alpha u^3)/u$ on noisy well-conditioned data is wrecked by the u'' term (which amplifies the noise by k^2), whereas the L-BFGS recovery V^* tracks V_* . Right: the noise added to the data u is invisible to the naked eye.

The penalty we add must match the structure of the unknown. We compare two choices, each weighted by β and added to the misfit: Tikhonov, $R_2(V) = \beta \|V'\|_2^2$, which rewards smoothness, and total variation (TV), $R_1(V) = \beta \|V'\|_1$, which rewards piecewise-constant structure with sharp edges. Which one wins depends entirely on the truth V_* .

For a smooth V_* (a cosine) Tikhonov matches the prior: sweeping β , it reaches an order of magnitude lower error than TV (Figure 9, Table 5), and at the optimal weight β^* it simultaneously lowers the error and despikes the null-point artifacts of Section 5. For a non-smooth V_* (a box/spike) the ranking flips: TV preserves the edges that Tikhonov smears out (Figure 10). The lesson is that regularization helps precisely when its penalty encodes the right structure of the unknown — it supplies the information the data cannot.



(a) Recovery vs. β .

(b) Despiking at β^* .

Figure 9: Smooth V_* (cos): Tikhonov reaches the lower error and reconstructs the smooth potential.

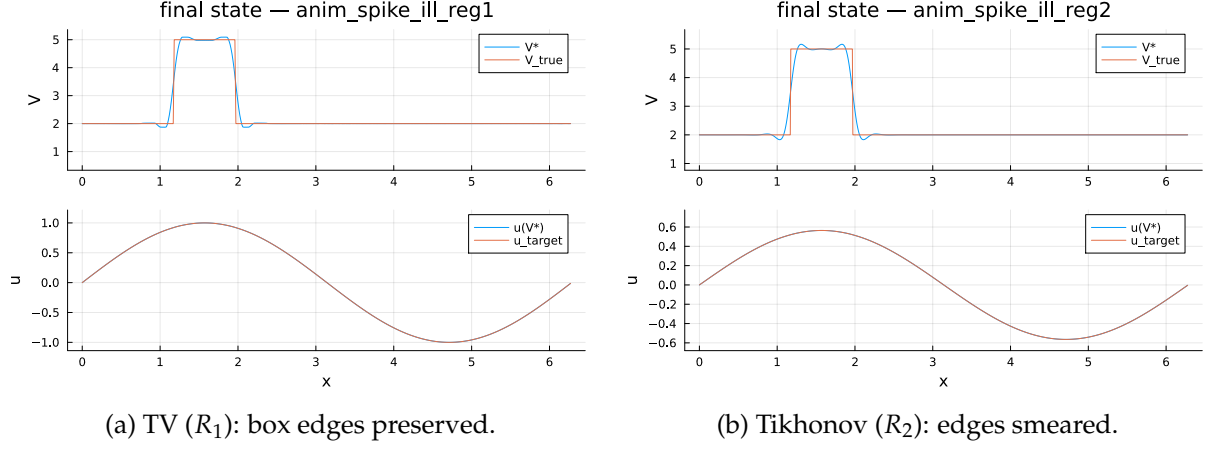


Figure 10: Non-smooth V_* (spike), converged V^* (top) and state $u(V^*)$ (bottom): the winner flips — TV preserves the edges, Tikhonov smears them.

method	β	misfit	pot_err
R1(TV)	3.35982e-8	3.14248e-8	0.00616089
R2(Tik)	3.35982e-8	3.12128e-8	0.00160662

Table 5: Optimal regularisation weight β^* (the value minimising the potential error over the β -sweep of Figure 9a), with the resulting misfit and recovery error, for the smooth V_* . Tikhonov (R_2) reaches an order of magnitude lower error than TV (R_1).

7 Conclusion

A Reproducibility

13th Gen Intel(R) Core(TM) i7-1360P (16 threads); Julia 1.12.6, BLAS threads 8; commit 6060a69, 2026-06-21.